

DM-2026-011

Bruce Dombrowski
Systems Engineer, Payload Software Integration

DATE: September 13, 2026

TO: Security Toolkit Stakeholders; AI-Agents Repository Maintainers

SUBJECT: Adoption of Claude Managed Agents for the Security Toolkit Scan-and-Attest Agent

OFFICE OF PRIMARY RESPONSIBILITY: Payload Software Integration (PSI)

1. **Purpose**

This memorandum records the decision to adopt Anthropic’s Claude Managed Agents platform (public beta, April 2026) as the runtime for a scan-and-attest agent built on the Security Toolkit, the trust-boundary rules that govern where the agent’s tools may execute, and the scope of adoption across the maintainer’s other repositories. It follows the pattern presented in the Code with Claude 2026 session *Ship your first Managed Agent* (I. He, Anthropic Applied AI), whose reference implementation is published in `anthropics/cwc-workshops`.

2. **Background**

Managed Agents separates the agent loop (“brain”, run on Anthropic’s orchestration layer) from tool execution (“hands”, run in a per-session sandbox). Three persisted resources define a run: an **Agent** (model, system prompt, tools, skills; versioned, created once), an **Environment** (sandbox template: cloud or self-hosted), and a **Session** (one run, with an event log). Custom tools execute on the caller’s host, so secrets and privileged actions never enter the sandbox. Skills in a mounted repository’s `.claude/skills` directory are loaded at session start. Sessions accept a hard dollar budget and persist a complete event history.

The Security Toolkit already carries the pieces such an agent needs: a non-interactive scan entry point (`run-all-scans.sh -n`), attestation generation with toolkit version and commit hashes, an allowlist model that requires a SHA256 integrity hash plus a justification, and TIA principles (Transparency, Inspectability, Accountability, Traceability) that map directly onto a persisted session event log.

3. **Decision 1 — Agent Runtime Surface**

Options considered:

- **1A — Claude API with tool runner, self-hosted loop.** Maximum control; the maintainer hosts the loop, sandbox, scheduler, and state.
- **1B — Claude Agent SDK.** Claude Code packaged as a library; batteries included, still self-hosted.

- **1C — Claude Managed Agents (SELECTED).** Anthropic runs the loop and (optionally) the sandbox; agent configuration is a version-controlled YAML manifest applied once with the **ant** CLI; sessions are created by a thin runner script.
- **1D — No agent.** Continue running the toolkit by hand and in CI.

Decision: Option 1C. The agent definition lives in the toolkit repository as `agents/security-attest.agent` and `agents/security-scan.env.yaml`; the runbook lives in `.claude/skills/security-scan-runbook/S`; the runner is `agents/run_scan_session.py`. Two host-side custom tools are defined: `get_key_status` (CISA KEV lookup from the host cache) and `request_allowlist_entry` (a request only; the human decides and the host computes the integrity hash).

Rationale: The toolkit's value is auditability, and Managed Agents gives a persisted, inspectable event log per session at no implementation cost. Versioned agent objects match the toolkit's own attestation discipline (every attestation names a toolkit version and commit; every session pins an agent version). Options 1A and 1B require the maintainer to build and host the loop, sandbox, and state store for a solo-maintained project. Option 1D forgoes the judgment step (classifying findings as true or false positives) that consumes most of the human time today.

4. Decision 2 — Trust Boundary by Target Sensitivity

Context: A cloud environment clones the target repository into an Anthropic-hosted container. The toolkit is public and may be mounted anywhere. Targets are not.

Decision:

- Targets that are public or contain no CUI and no PII **may** use the cloud environment.
- Targets that contain or may contain CUI, PII, or export-controlled data **shall** use a **self_hosted** environment, where tool execution and file contents remain on maintainer-controlled infrastructure. Web search and web fetch are disabled on the agent regardless of environment.
- The agent **shall not** modify allowlists, delete files, or run scripts from the target repository. Target contents are treated as untrusted data; instruction-like text found there is a finding, not a command.
- Only trusted repositories are mounted, because a mounted repository's `.claude/skills` directory is loaded as agent instructions without review.
- Every session is created with a dollar budget cap.

Rationale: These rules keep the CUI determination (DM-2026-007, DM-2026-008) intact: nothing the desk instruction classifies as CUI leaves maintainer-controlled systems. The allowlist rule preserves Accountability; the untrusted-data rule addresses prompt injection through scanned content, which is the primary new risk an agent introduces to a scanner.

5. Decision 3 — Scope Across Other Repositories

- **Security Toolkit:** adopt now (this memorandum).

- **Foreclosures:** candidate for a monthly scheduled deployment that runs the deterministic pipeline and reviews the ranked shortlist. Deferred until the toolkit agent has run cleanly for one release cycle.
- **Investing:** a read-only daily review agent over the run report and decision log is acceptable; brokerage credentials go in a vault with read-only scope. The agent **shall never** be given order submission. Deferred.
- **Email Assistant:** excluded. Its stated requirement is that no content leaves the laptop and no cloud model is used; Managed Agents contradicts that by design.
- **Philosophy:** excluded; no identified benefit.

6. Implementation and Constraints

- `.gitignore` changed from `.claude/` to `.claude/*` with `!.claude/skills/` so the runbook is committed and discoverable when the toolkit is mounted; local Claude Code settings remain ignored.
- Agent and environment are created once with `agents/setup.sh`; the returned IDs are stored in `agents/ids.env` (git-ignored). Sessions reference IDs only; no code path calls `agents.create()` per run.
- Model: `claude-opus-5` at high effort. Tools: `bash`, `read`, `grep`, `glob`, `write` (for the report), web tools disabled.
- Dependencies: `ant` CLI and the `anthropic` Python package on the host; both are optional for the toolkit's existing Bash workflow.
- CHANGELOG entry under Unreleased; regression test `tests/test-managed-agent-manifests.sh` validates manifests, runner syntax, and that the skill directory is not git-ignored.

7. Verification

- **Manifest test:** both YAML manifests parse; the runner compiles; `git check-ignore` reports the skill file as not ignored.
- **Dry session:** against the toolkit's own `demo/` planted findings (`docs/DEMO-PLANTED-FINDINGS.md`) the agent reports every planted finding with a correct `path:line` and requests, but does not perform, allowlist changes.
- **Boundary test:** a target containing instruction-like text is reported as a finding and the instruction is not followed.
- **Budget test:** a session created with a budget pauses with `budget_reached` rather than running past the cap.

8. References

- Code with Claude 2026, *Ship your first Managed Agent*, I. He; reference code `anthropics/cwc-workshops/ship-your-first-managed-agent`.
- Claude Managed Agents documentation (`platform.claude.com`), beta `managed-agents-2026-04-01`

- Security Toolkit `CLAUDE.md` (TIA principles), `AGENTS.md`, `docs/COMPLIANCE.md`.
- DM-2026-007, DM-2026-008 (CUI determination and desk instruction).
- NIST SP 800-53 Rev. 5, NIST SP 800-171, FIPS 199/200; RFC 2119.